

6D Object Pose Tracking with the Deep Iterative Matching Network

Lirui Wang

Abstract—Pose estimation has been studied extensively by computer vision and robotics researchers. While direct pose regression from RGB image is limited by various factors, DeepIM (Deep Iterative Matching) [1] takes an iterative approach to refine initial pose estimation by matching rendered image against real image. This project mainly addresses the performance of DeepIM on YCB dataset and tracking tasks.

I. INTRODUCTION

In robotic manipulation and virtual reality tasks, localizing objects in 3D from images has played a critical role. While depth cameras can be used for object pose estimation, such cameras have limitations with respect to frame rate, field of view, resolution, and depth range. Two traditional computer vision pipelines to tackle pose estimation involves heavy feature engineering to estimate key points of the object and convert the 6D pose estimation problem into a pose classification problem by discretizing the pose space. However, those methods are not robust to small errors and not generalized well to unseen objects. DeepIM, a FlowNet backbone deep network is introduced with an untangled representation of the SE3 transformation between object poses to achieve accurate pose refinements. This work does extensive experiments of DeepIM on YCB dataset [2] and have the following discussions: Randomly generating poses on synthetic training can help avoid overfitting on training set poses. Generating realistic occlusion on synthetic images can help network's performance on real occluded scene. We also worked on extending the work to tracking tasks, and achieves good results.

II. DEEPIIM

A. Zoom-In

In order to obtain enough details from RGB image, a zoom-in operation for rendered and observed image is performed before feeding them into the network. The structure zooms in and performs bilinear up-sampling to achieve the same size as the original image while keep the aspect ratio of the object unchanged. This spatial transformation provides the network with more image signals and information about the center of the object.

B. Network Structure

DeepIM uses a FlowNetSimple backbone [3] to predict a relative SE3 transformation to match the observed and rendered image of an object in an iterative manner. Specifically, in the i th stage of the iterative matching, given a 6D pose estimate $p^{(i-1)}$ from the previous step, we render a synthetic

image using the 3D object model viewed according to $p^{(i-1)}$. Additional flow losses improve stability during training.

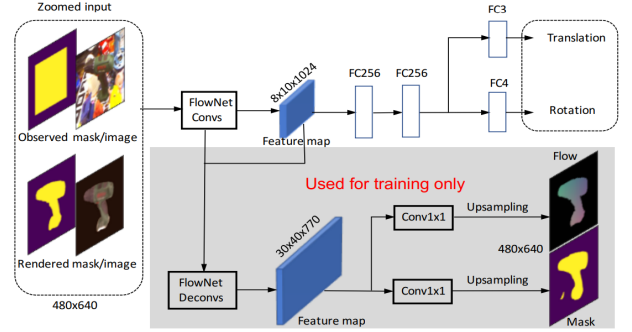


Fig. 1: DeepIM Network Structure

C. Untangled Transformation Representation

The method chooses the coordinate frames such that 3D object translations correspond to pixel-wise translations in the image plane along with a change in scale, and 3D rotations correspond to rotations around the image center along with a pan and tilt of the image. To estimate the relative translation, let $t_{tgt} = (x_{tgt}, y_{tgt}, z_{tgt})$ and $t_{src} = (x_{src}, y_{src}, z_{src})$ be the target translation and the source translation. The network is trained to regress to the relative translation $t_{\delta} = (v_x, v_y, v_z)$, where v_x and v_y denote the number of pixels the object should move along the image x-axis and y-axis and v_z is the scale change of the object:

$$\begin{aligned} v_x &= f_x(x_{tgt}/z_{tgt} - x_{src}/z_{src}) \\ v_y &= f_y(y_{tgt}/z_{tgt} - y_{src}/z_{src}) \\ v_z &= \log(z_{tgt}/z_{src}) \end{aligned}$$

where f_x and f_y denote the focal lengths of the camera.

D. Point Matching Loss

A modified version of the geometric reprojection loss call Point Matching Loss is proposed: Given the ground truth pose $p = [R|t]$ and the estimated pose $\hat{p} = [\hat{R}|\hat{t}]$, the point matching loss is computed as

$$L_{pose}(p, \hat{p}) = \frac{1}{n} \sum_{j=1}^n L_1((Rx_j + t) - (\hat{R}x_j - \hat{t}))$$

where x_j denotes a randomly selected 3D point on the object model and n is the total number of sample points used in computing the loss.

III. EXPERIMENTS AND TRACKING

Besides working on the pytorch implementations of DeepIM, we make a few ablation studies on the network. In the training procedure, real image provides a signal in real world for the network and synthetic data arguments the training data and help the network avoid overfit. We control the synthetic ratio to be 5, meaning that for one epoch, we expect to see $\frac{1}{6}$ of the real images and the rest should be synthetic images. In terms of the video split, we sample around 1000 images from the first 6 videos containing the object for training, and leave the rest videos for testing. More training and testing details about the network can be found in the original paper and their experiments.

The $5^\circ, 5cm$ metric considers an estimated pose to be correct if its rotation error is within 5° and the translation error is below 5cm. $6D$ Pose uses the average distance metric to compute the averaged distance between points transformed using the estimated pose and the ground truth pose. A pose estimation is considered to be correct if the computed average distance is within 10% of the model diameter. $2D$ Projection focuses on the matching of pose estimation on 2D images. We compute the error and accept a pose estimation when the 2D projection error is smaller than a threshold of 5 pixels.

A. Synthetic Pose Distribution

During the experiments, we found that the distribution of the ground truth poses of the synthetic image heavily affect the generalization of the model: (Perturb: The poses are sampled from real image poses with small perturbation, Random: The poses are randomly generated within certain bounds). The expected rotation distribution should cover a sphere if we are rotating a vector in z direction with unit length. However, the perturbation poses only cover a small corner of the sphere, so its performance is heavily affected by overfitting on the ground truth training poses.

TABLE I: Comparison between Different Training Poses

Metric	$5^\circ, 5cm$	6D Pose	2D Projection
Perturb	0.5432	0.6391	0.4427
Random	0.8839	0.9793	0.6328

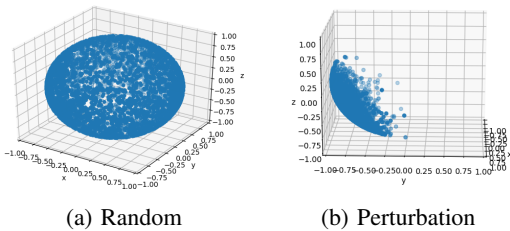


Fig. 2: Pose Distribution Visualization

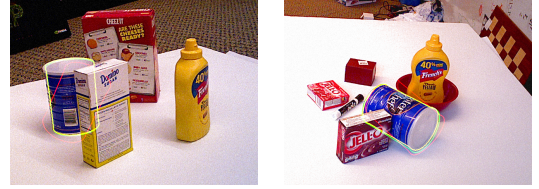


Fig. 3: Performance on YCB Video Dataset

B. Occlusion and Synthetics

In this section, we trained the network on synthetic images only, as this is our main interests to offset the lack of real images dataset by generating synthetic images. Moreover, we provide a new approach with controllable occlusion to generate poses for occluder using the 3D bounding boxes. Note that the computation time of this method does not differ from the naive way of perturbing translation and rotation that much. The method for generating occluder poses is as follows:

Occlusion Algorithm (Object BB_{3d} , Occluder BB'_{3d})

$BB_{2d} \leftarrow$ Smallest bounding box of BB_{3d} projection

$BB'_{2d} \leftarrow$ Sample a random corner of BB_{2d} to occlude

Match the opposite corner of BB'_{3d} to BB'_{2d}

pose of Occluder $\leftarrow$ solvePNP for BB'_{3d} and BB_{2d}

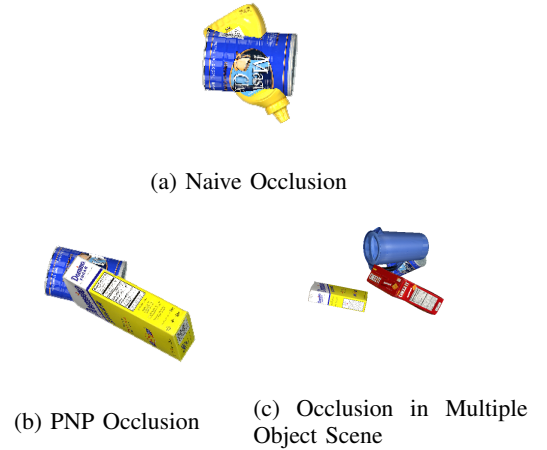
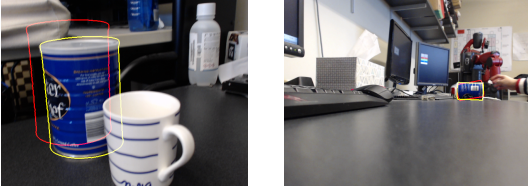


Fig. 4: Generated Occlusion Images

C. Tracking

The DeepIM structure strives for refining poses based on an initial pose, so one intuitive usage is to apply on tracking. At real time, DeepIM network runs at 12 fps per object using the NVIDIA 1080 Ti GPU with 1 iterations. For tracking, instead of randomly perturb the ground truth poses as initialization, we use the pose from the previous time step as an initialization for the next step's refinement. Note that the transformation between any two steps are not likely to be above the perturbation during training and testing. So, even as a pose refinement method based on previous estimation, in most cases it still has the advantage of an one-shot pose estimation method: prediction at current

time step is unlikely to be affected by the previous drift. In addition, the tracking works well without acquiring the actual camera intrinsics. This confirms that all information that the network processed are in image plane. Recall that the translation prediction from the network is based on pixel translation and size ratio. The image matching idea gives a very robust pose tracking with respect to light condition, occlusion, pose initializing method and pose ranges. We tested edge cases such as occlusion center, fast rotation, and farther distance and the network performs constantly well. To illustrate that, we provide experiment results comparing tracking and testing performance for multiple objects using DeepIM. Demo tracking videos on the test set is attached in the link: https://drive.google.com/file/d/1fxJajH_ve-NSW2pcdq0w6Wv4ngyUOPLV/view?usp=sharing. A real time video is in https://drive.google.com/open?id=1byH4d2Ud8ThnrOnVv75N9Rv9k_tJxclX.



(a) Real Time Tracking Im- (b) Farther and Laid Flat Pose
age

IV. CONCLUSION AND FUTURE WORK

In this work, we present some extended works and experiments of DeepIM on YCB dataset. This concludes that DeepIM would also work well on objects of daily lives. For future works, we attempt to train the network to robustly predict poses, on a par with RGBD and ICP method, based on synthetic training data only. In addition, we would also like to improve the speed and performance of the tracking applications of DeepIM. More precisely, to work on tracking, we will expect to train a network with better initial pose estimates and more accurate refinements. This tracking task can also be extended to robot arm tracking and other robotics manipulation tasks.

V. APPENDIX

TABLE II: Object 002 Master Chef Can

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.3692	0.7713	0.2317
Reinitialize Pose	0.3792	0.7509	0.2541
PoseCNN Result	0.0427	0.1024	0.0010

TABLE III: Object 004 Sugar Box

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.5646	0.7458	0.4416
Reinitialize Pose	0.4184	0.5811	0.2584
PoseCNN Result	0.1362	0.4433	0.0711

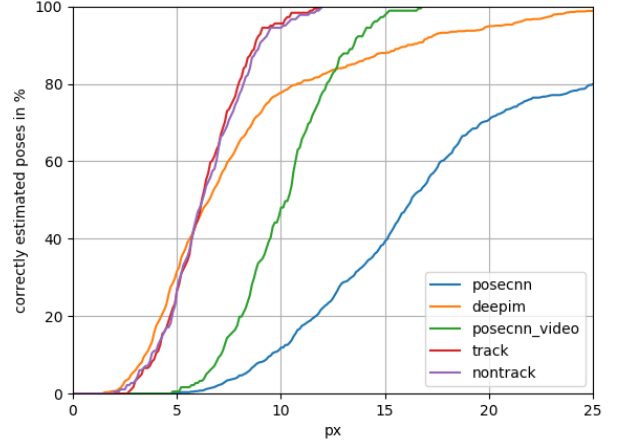


Fig. 6: Master Chef Can 2D Reprojection Performance Plot

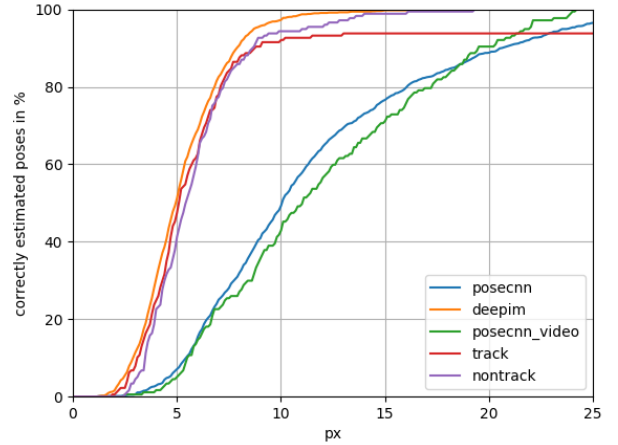


Fig. 7: Sugar Box 2D Reprojection Performance Plot

TABLE IV: Object 010 Potted Meat Can

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.4024	0.8669	0.6065
Reinitialize Pose	0.3757	0.8136	0.5888
PoseCNN Result	0.1061	0.4273	0.4182

TABLE V: Object 006 Mustard Bottle

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.7993	0.9831	0.6474
Reinitialize Pose	0.8113	0.9717	0.6588
PoseCNN Result	0.1933	0.7647	0.0644

TABLE VI: Object 035 Power Drill

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.3718	0.7869	0.0940
Reinitialize Pose	0.2768	0.7561	0.0856
PoseCNN Result	0.0889	0.4201	0.0331

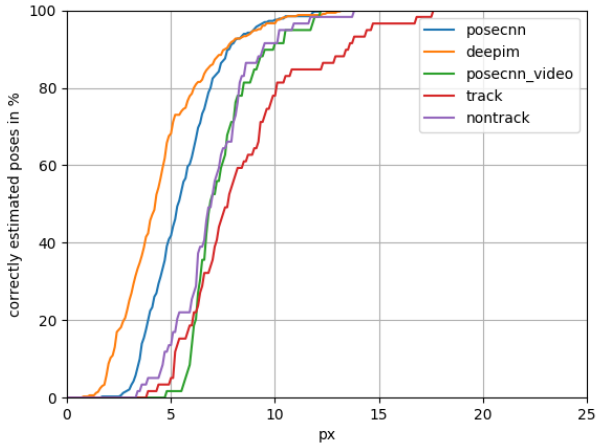


Fig. 8: Potted Meat Can 2D Reprojection Performance Plot

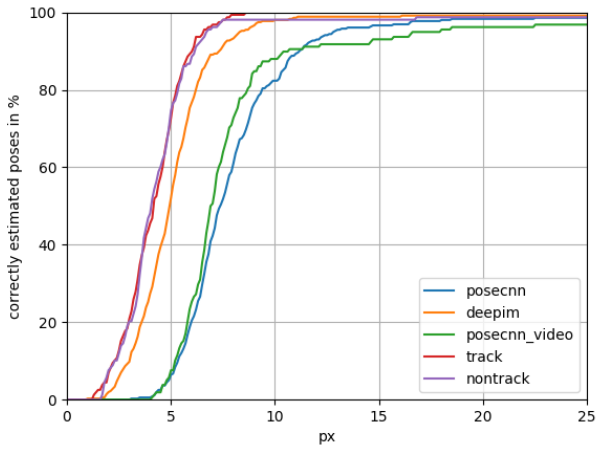


Fig. 9: Mustard Bottle 2D Reprojection Performance Plot

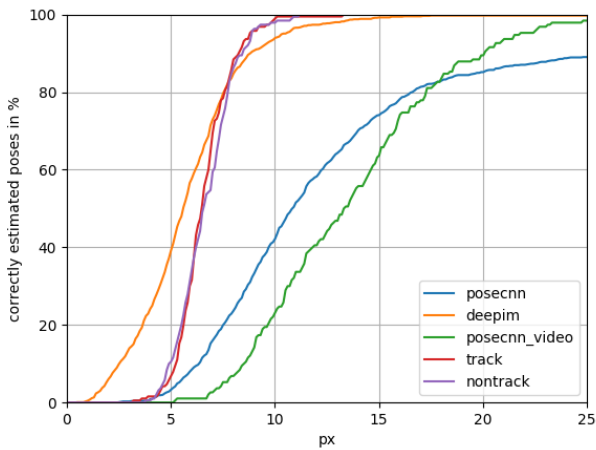


Fig. 10: Power Drill 2D Reprojection Performance Plot

TABLE VII: Object 003 Cracker Box

Metric	5°, 5cm	6D Pose	2D Projection
Tracking	0.3718	0.7869	0.0940
Reinitialize Pose	0.3919	0.4981	0.0746
PoseCNN Result	0.0276	0.2846	0.0012

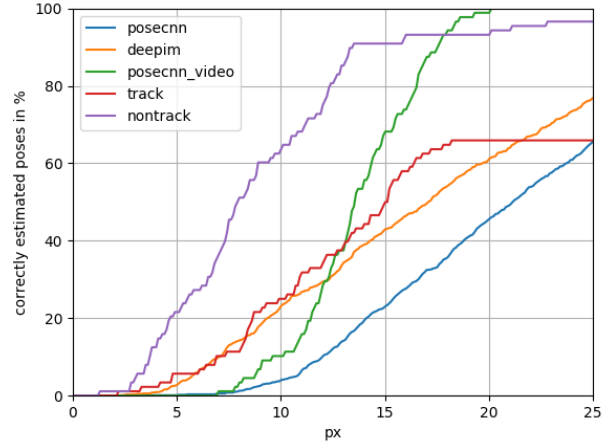


Fig. 11: Cracker Box 2D Reprojection Performance Plot

REFERENCES

- [1] Y. Li, G. Wang, X. Ji, Y. Xiang, and D. Fox. Deepim: Deep iterative matching for 6d pose estimation. arXiv preprint arXiv:1804.00175, 2018.
- [2] Xiang, Y., Schmidt, T., Narayanan, V., Fox, D.: PoseCNN: A convolutional neural network for 6D object pose estimation in cluttered scenes. arXiv preprint arXiv:1711.00199 .
- [3] Dosovitskiy, A., Fischer, P., Ilg, E., Hausser, P., Hazirbas, C., Golkov, V., van derSmagt, P., Cremers, D., Brox, T.: FlowNet: Learning optical flow with convolutional networks. In: IEEE International Conference on Computer Vision (ICCV). (2015) 27582766